COS40003: Concurrent Programming

Report 1: Learning Summary (Weeks 1–4)

Dang Duc Minh
Student ID: 105559691

I. Week 1 - Introduction

The summary below for Week 1 covers **Part I** of **Lab 1**.

A. Concurrent Computing

Concurrent computing is a fundamental computing paradigm that means multiple computational tasks are being executed on a same time period, but not necessarily simultaneously. Instead of processing tasks sequentially, the OS (operating system) will manages several execution flows at once to maximize resource utilization.

Concurrent computing relies heavily on **time-sharing**. The OS divides the CPU into small execution flows and rapidly switches between active processes. This approach fundamentally maximizes system effciency by allowing processes that are ready to be executed to be processed continously.

Coordinating concurrent tasks requires addresing two fundamental challanges:

1) **Scheduling:** Determining which process executes next to ensure fairness and response time.
2) **Concurrency Control:** Managing shared resources to prevent **race conditions** (inconsistent data or state caused by timing issues) and **deadlocks** (processes that wait indefinitely where two or more threads or processes wait indefinitely for each other, which prevents any of them from making progress).

A common example of concurrent computing is a web browser (e.g., Chrome, Safari) is opening multiple tabs at one, where each tab are doing seperate tasks such as playing a video, downloading a file, or fetching live chat messages.

According to Rob Pike, co-creator of the Go programming language: **"concurrency is about dealing with lots of things at once, while parallelism is about doing lots of things at once"** [1].

B. Parallel Computing

Parallel computing is a computing architecture in which multiple calculations or execution flows are carried out at the exact same time. Concurrent computing often creates an illusion of simultaneous progress on a single CPU from the user's perspective. Parallelism, on the other hand, represents multiple executions at the exact same time, as illustrated in Figure 1.

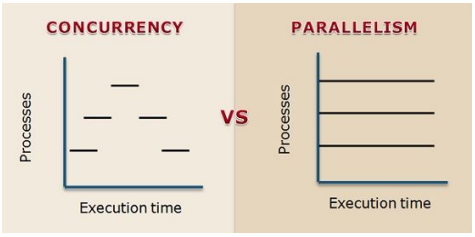


Fig. 1. Concurrent execution vs. Parallel execution [2].

In parallel computing, a large computational workload is broken down into smaller, independent subtasks that can be executed concurrently. Each subtasks are executed simultaneously on seperate processors, and once they are finished, the individual sub-results are merged to produce the final output. In a typical parallel computing architectures, processors communicate directly through **shared memory** to ensure low-latency data exchange across processes.

A real-world example of parallel computing is a **Graphics Processing Unit (GPU)**, which comprises thousands of smaller arithmetic cores for executing identical operations such as rendering graphics or computing matrix multiplications using the **Single Instruction, Multiple Data (SIMD) architecture** (shown in Figure 2).

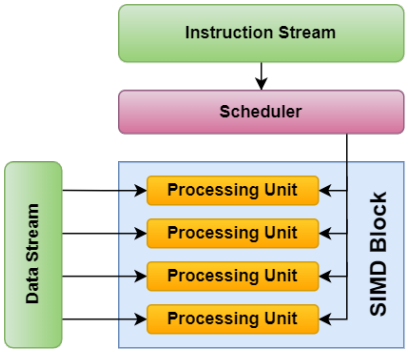


Fig. 2. SIMD Architecture [3].

C. Distributed Computing

Distributed computing consists of multiple autonomous computing entities that are connected through a shared network to solve complex computational problems. A fundamental architectural distinction between parallel computing and distributed computing lies in memory organization. In parallel

computing, processors shares a common memory space to exchange information directly. But for distributed computing, each individual computer cannot access each other's memory directly and must exchange information through **message passing** (communication protocols or middleware) [4] (shown in Figure 3).

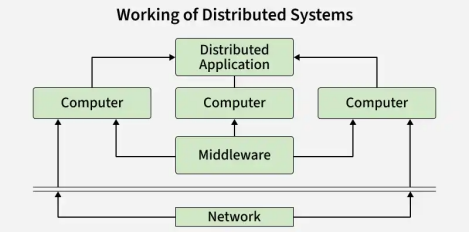


Fig. 3. Architecture of a Distributed System [4].

Distributed computing architectures are commonly classified into two primary paradigms:

1) **Cluster computing:** Composed of **homogeneous** machines, which feature similar hardware configurations and identical OSs, that are **co-located** within a single area. This architecture is designed for high availability and high-performance computing.
2) **Grid computing:** Comprised of **heterogeneous** computing entities that are **geographically dispersed** across different physical locations. Grid computing represents the conceptual predecessor to modern cloud infrastructure.

A real-world example that implements both cluster and grid computing is the **Worldwide LHC Computing Grid (WLCG)** [5]. Local server clusters are deployed for high-performance batch processing, and these clusters are linked globally into a centralized grid to analyze high volumes of data.

D. Cloud Computing

Cloud Computing is one of the most demanded computational infrastruture in modern computing, delivering services such as servers, storage, databases, and networking over the Internet.

The fundamental goal of cloud computing is abstraction, which hides the underlying physical hardware and maintenance complexity from users, allowing them to access on demand resources with seamless scalability and minimal configuration. Behind this abstraction, cloud computing relies extensively on distributed systems since it coordinates massive networks of server clusters to ensure high availability and data redundancy across multiple regions.

Cloud services are organized into three primary architectural models:

1) **Infrastructure as a Service (IaaS):** Provides virtualized computing infrastructure, including virtual machines, block storage, and virtual networks (e.g., AWS EC2, Microsoft Azure VMs)
2) **Platform as a Service (PaaS):** Provides a managed execution environment and development framework where developers can build and run applications without configuring the underlying OS or runtime (e.g., Vercel, Firebase)
3) **Software as a Service (SaaS):** Provides software applications that can be directly accessed via web browsers (e.g., Microsoft 365, Google Docs).

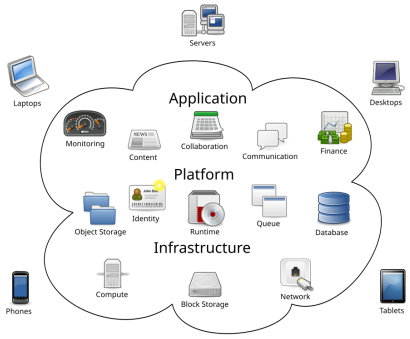


Fig. 4. Cloud Computing Logical Architecture and Service Models [6].

E. Fog and Edge Computing

Fog and Edge computing designed to bring processing and data storage closer to data-generating sources, primarily Internet of Things (IoT) devices. These computing paradigms address the limitations of address critical limitations of cloud computing, such as transmission latency and bandwidth bottlenecks.

- **Edge Computing:** Performs computation directly on local hardware devices such as microcontrollers, sensors, and local gateways.
- **Fog Computing:** Extends the cloud computing paradigm toward the physical local environment by introducing an intermediate layer that coordinates compute resources between local devices (nodes) and the remote cloud.

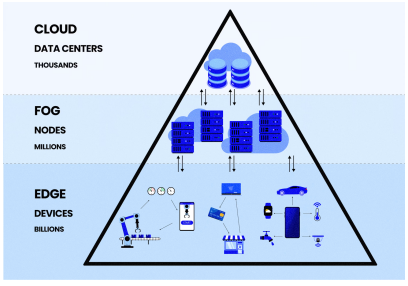


Fig. 5. Edge Computing Architecture and Data Processing Flow [8].

A real-world implementation that utilizes both paradigms is smart traffic management in intelligent transportation systems [7]. Vehicle sensors process immediate collision data

locally at the edge devices, while local base station units operate as an intermediate fog layer nodes to coordinate traffic flow and signal timings in real time.

Lab 1 - Part II

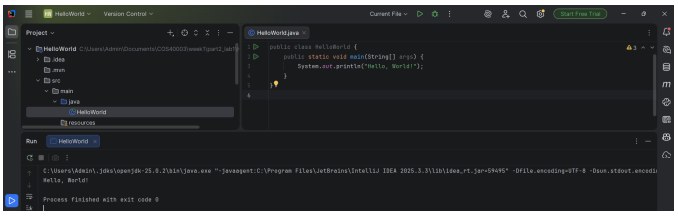


Fig. 6. The "Hello World!" Java application compiled and ran successfully.

II. Week 2 - Process: Concept and API

The summary below for Week 2 covers **Part I** of **Lab 2**.

A. Process Definition

A process is essentially a program that being executed. When a process is running, the OS allocates an address space in RAM (containing code, data, heap and stack) and the CPU read the instructions and execute them on by one.

The differences between a process and a program lies in their execution state. A program is stored as an executable binary on the disk, wheras a process is an active instance executing dynamically in memory.

B. The Need for Processes

Early computing executed single programs one at a time, which creates severe bottlenecks whenever a task had to wait for slow I/O operations, which forces the processor to idle. The abstraction of process was invented to solve this inefficiency, allows the OS to safely execute multiple programs concurrently with isolated memory.

The famous computer scientist Edsger W. Dijkstra has demonstrated the necessity of process through showing that breaking complex computing workloads into separated sequential execution units is essential to keep the CPU continuously utilized while preventing tasks from overwriting shared data [10].

C. Process Operations

1) Process Components: A process comprises of four essential elements:

- **Image of a program:** The compiled code, environment, and dependencies.
- **Memory (Address Space):** An isolated virtual address space allocated to each running process (shown in Figure 7). The program instructions data and static data remain fixed throughout execution, while two dynamic memory segments grow towards each other:

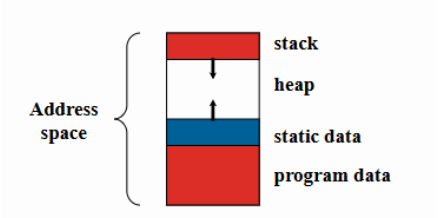


Fig. 7. Process Address Space Structure [9].

  - **Stack:** Managed automatically by the CPU in Last-In, First-Out (LIFO) order to store variables, function parameters, and return addresses. It grows downward toward lower memory addresses.
  - **Heap:** Managed at runtime for for dynamic memory allocation. It grows upward toward higher memory addresses.
- **CPU state:** Registers, Program Counter (PC), and Stack Pointer (SP).
- **OS state:** Metadatas such as opened files, temporal statistics, etc.

2) Context Switch: **Context switch** is a mechanism that alternates the CPU between different processes to enable multitasking. Because a single CPU core can execute only one process at a time, the OS rapidly switches active execution contexts between processes (represented as programs `Prog1` and `Prog2` in Figure 8) across four phases:

1) **Phase 1 (`Prog1` Running):** `Prog1` executes instructions on the CPU, while `Prog2` remains suspended.
2) **Phase 2 (Save State):** The OS saves the CPU registers, PC, SP of `Prog1`, placing it into suspension.
3) **Phase 3 (Restore State):** The OS restores the previously saved registers, PC, and SP belonging to `Prog2`.
4) **Phase 4 (`Prog2` Running):** `Prog2` takes over the CPU and resumes execution, while `Prog1` waits in memory.

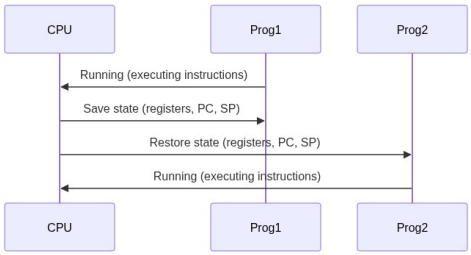


Fig. 8. Process Context Switching Mechanism Between Two Processes.

3) The Process Abstraction: The **process abstraction** is a conceptual bridge between shared physical hardware and private program execution processes:

- **Physical View:** One physical CPU shared and context-switched among processes.
- **Logical View:** Each process operates as if it has its own dedicated CPU and memory.

- **Time-Sharing:** Processes rapidly take turns using the CPU, enabling multi-computational processes on a single core.

4) Process States: A process transitions through three primary execution states:

- **Ready:** The process is loaded in memory and prepared to run.
- **Running:** The process currently holds the CPU and is actively executing instructions.
- **Blocked:** The process has released control of the CPU to wait for an event (such as I/O completion) and cannot execute until that event happens.

Transitions between these states occur as follows:

- **Ready** → **Running:** The scheduler picks the process to run.
- **Running** → **Ready:** The scheduler suspends the process.
- **Running** → **Blocked:** The process waits for an event (e.g., I/O request).
- **Blocked** → **Ready:** The event occurs (e.g., I/O completes).

D. Process APIs

There are three fundamental process management APIs in Unix systems.

1) `fork()` **- Creating a Process by Duplication:** Creates an exact clone of the caller with its own memory space. It returns `0` to the child process and the child's Process ID (PID) to the parent, allowing both to branch into different execution paths.
2) `wait()` **- Synchronizing with Child Termination:** Pauses the calling parent until one of its child processes finishes execution. It returns the terminated child's PID and collects its exit status, preventing zombie processes (dead processes waiting for their parents).
3) `exec()` **- Replacing Current Program Code:** Replaces the process's current memory contents (code, data, heap, stack) with a new executable program from disk. The process retains its original PID but executes the new code.

Lab 2 - Part II

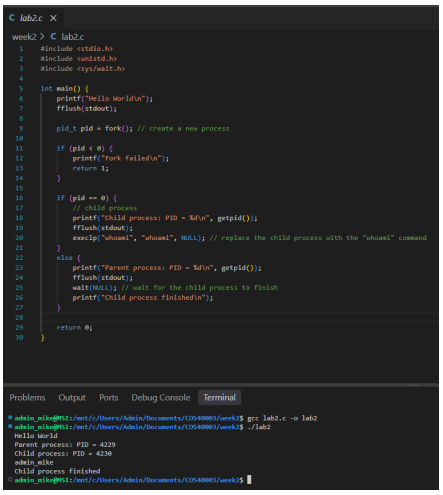


Fig. 9. Example Hello World C Program using `fork()`, `wait()`, and `exec()` APIs.

III. Week 3 - Scheduling

The summary below for Week 3 covers **Part I** of **Lab 3**.

A. CPU Scheduling

The OS must decide which process runs next on the CPU. This decision directly impacts system performance and the user experience.

There are several **CPU Scheduling Criterias** that the OS can use:

- **CPU Utilization:** The total time of the CPU being busy executing processes.
- **Turnaround Time:** The total time of a job submission to its termination.
- **Response Time:** The amount of time it takes to get the first response.
- **Missed Deadlines:** The number of jobs that fail to complete before their deadlines.

Beyond these performance criteria, there are several **CPU Scheduler Options** that define how scheduling policies are governed:

- **Priorities:** Determines which process will be executed based on its priority level.
- **Preemptive vs. Non-preemptive:**
  - **Preemptive:** The OS can interrupt a running task at any time to take over.
  - **Non-preemptive:** The OS will not interrupt a running task until it finishes its execution.

B. First-Come, First-Served (FIFO)

The **First Come, First-Served algorithm** takes the first process that arrives and executes it until completion. Processes that comes next will wait in a queue until the CPU is avaliable (FIFO queue).

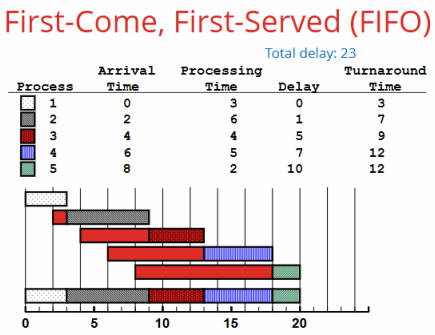


Fig. 10. FCFS Scheduling Example [11].

As shown in Figure 10, five processes are scheduled non-preemptively based on their arrival times. Process 1 arrives at $t = 0$ and runs immediately, while subsequent processes must wait in the ready queue until the running task completes. Shorter processes that arrive later suffer significant waiting time such as Process 5, which requires only 2 units of processing time but experiences a delay of 10 units, which results in a high cumulative delay of 23 units.

C. Shortest Job First (non-preemptive)

The **Shortest Job First (SJF) algorithm** selects the process with the shortest estimated execution time from the ready queue. This approach is non-preemptive, so that once a process starts executing, it will run to completion before the next process is selected.

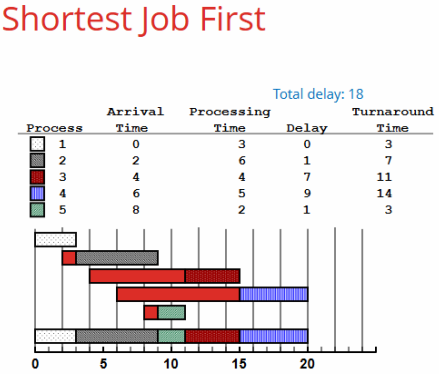


Fig. 11. SJF Scheduling Example [11].

As shown in Figure 11, after Process 1 and Process 2 complete at time unit 9, Process 5 is scheduled next because its required processing time is only 2 units, compared to 4 units for Process 3 and 5 units for Process 4. Prioritizing shorter jobs significantly decreases waiting time, bringing the total delay down to only 18 units.

D. Shortest Job First (with preemption)

The **Shortest Job First with preemption** also known as **Shortest Remaining Time (SRT)**. Whenever a new process arrives in the ready queue, the scheduler compares its total processing time against the remaining time of the currently running task. If the new process has a shorter remaining burst time, the OS will preempt the running process by moving it back to the ready queue, then immediately allocates the CPU to the new process.

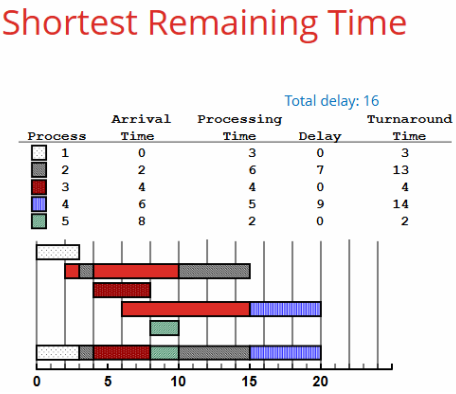


Fig. 12. SRT Scheduling Example [11].

As shown in Figure 12, at time unit 4, Process 3 preempts Process 2 because its required 4 units of processing time is shorter than the 5 units remaining for Process 2. Prioritizing shorter jobs immediately upon arrival allows Process 3 and Process 5 to finish with zero delay, reducing the total delay to 16 units.

E. Round-Robin Scheduling

The **Round-Robin (RR) Scheduling algorithm** limits the amount of time a process can run on the CPU to a fixed time slice (quantum). If the process does not finish within its allocated time, it is preempted for the next process in the queue. This approach ensures that all processes receive fair amount of CPU time.

The effectiveness of Round-Robin Scheduling depends on the the number of jobs and the size of the time quantum. Large number of jobs means that the time between scheduling of a single job increases, which inherently increases the waiting time. While larger time quantum means that the time between the scheduling of a single job also increases, which also increases the waiting time. However, smaller time quantum would leads to better response time but also increases overhead due to more frequent context switches.

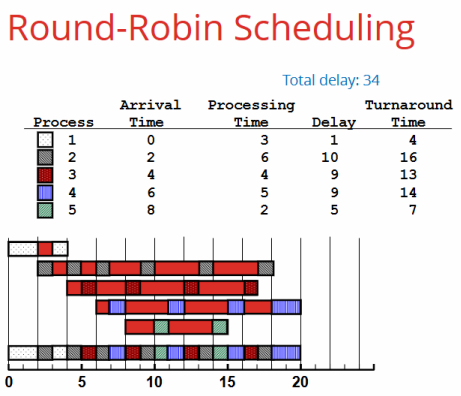


Fig. 13. Round-Robin Scheduling Example [11].

As shown in Figure 13, with a time quantum of 1 unit, the CPU continuously cycles through active jobs in the queue to grant each process immediate execution. Because execution is fragmented across different time slices, longer processes take much longer to reach final completion, which results in a high cumulative delay of 34 units.

F. Lottery Scheduling

The **Lottery Scheduling algorithm** is a type of proportional share scheduling in which the scheduler assigns a certain number of lottery tickets to each process. To select the next process to run, the scheduler randomly generates a winning ticket number, and the process holding the matching ticket receives CPU execution time.

For example, if there are 100 total tickets and Process A holds 50 tickets, Process B holds 15 tickets, and Process C holds 35 tickets, then Process A has a 50% chance of being selected, Process B has a 15% chance, and Process C has a 35% chance.

G. Multi-Level Feedback Queue (MLFQ)

The **Multi-Level Feedback Queue (MLFQ) algorithm** is designed to achieve two goals at once:

1) **Optimize Turnaround Time:** Algorithms like SJF or SRT are great for this, but we don't know the job lengths.
2) **Minimize Response Time:** Algorithms like RR are great for this, but produce terrible turnaround time.

MLFQ uses multiple queues with different priority levels. There are several rules that govern how processes are scheduled and moved between queues:

Rule 1: If Priority$(A) >$ Priority(B), A runs and B doesn't.

Rule 2: If Priority$(A) =$ Priority(B), A and B run in RR.

Rule 3: When a job enters the system, it is placed at the highest-priority queue.

Rule 4a: If a job uses up an entire time slice while running, its priority is reduced.

Rule 4b: If a job gives up the CPU before the time slice is up, it stays at the same priority level.

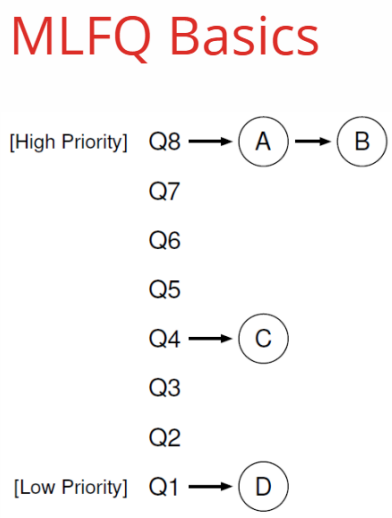


Fig. 14. Multi-Level Feedback Queue Example [11].

As shown in Figure 14, processes are distributed across priority tiers from Q8 to Q1. The scheduler selects jobs from the highest queue, executing Process A and Process B in RR at Q8 while Process C (at Q4) and Process D (at Q1) wait.

Lab 3 - Part II

What are the main tasks of the scheduler? Allocating CPU time to jobs, measuring time slices, assigning job priorities, and preempting running tasks when necessary.

What is the difference between preemptive and non-preemptive scheduling? Preemptive scheduling allows the operating system to actively interrupt and suspend running processes, while non-preemptive scheduling allows processes to run until they complete.

What is the advantage of a shorter scheduling quantum? It reduces response time because there is a shorter delay time between execution turns.

What is the advantage of a longer scheduling quantum? It results in fewer context switches, which reduces system overhead.

Why is feedback scheduling useful for interactive jobs? Interactive tasks are usually much shorter and frequently start and stop. Because feedback scheduling assigns every new task the highest priority, interactive jobs are run at the highest priority.

Are these scheduling policies subject to starvation?

- **Shortest Job First scheduling?** Yes, longer jobs can starve if shorter jobs continuously arrive.
- **Round Robin scheduling?** No, every job is guaranteed CPU time within a bounded time quantum cycle.
- **First Come First Served scheduling?** No, jobs are processed in strict arrival order, so every job will eventually execute.
- **Fixed Priority scheduling?** Yes, lower-priority jobs will starve if higher-priority jobs keep occupying the CPU.
- **Multi-level feedback scheduling?** No, because priority boost moves all jobs back to the highest-priority queue periodically.

IV. Week 4 - Thread

The summary below for Week 4 covers **Part I** of **Lab 4**.

A. The Need for Threads

The invention of processes has enabled OS-level multitasking, but it is not enough for subtasks within a single application due to heavy context-switch costs, data redundancy, and expensive Inter-Process Communication (IPC). Therefore, thread was introduced to enable concurrent subtasks within a shared address space.

According to benchmark measurements published by the Lawrence Livermore National Laboratory, creating 50,000 processes required 8.1 seconds while creating the same amount of threads only took 0.9 seconds on an Intel Xeon CPU [12]. This comparision demonstrated that thread creation requires lower operating system overhead than process creation.

B. Thread Definition

Threads are running subtasks of a process that execute instruction sequences concurrently within the same virtual address space. Threads have their own CPU context for independent control flow:

- **Program Counter (PC):** Tracks the address of the next instruction to execute.
- **Stack Pointer (SP) & Stack (memory):** Manages local variables, function calls, and return addresses.
- **Register States (CPU):** Holds temporary values and states for execution

Unlike a traditional process (single-threaded), which contains only a single stack for one execution flow, a multi-threaded process can execute multiple instruction streams simultaneously while sharing the same program code and heap memory.

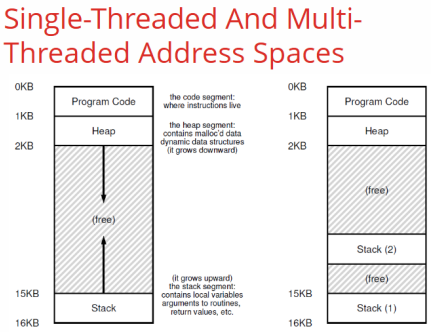


Fig. 15. Single-Threaded vs. Multi-Threaded Address Spaces [13].

Thread States:

- **Private Per-Thread State:** Elements that define the state of a particular flow of control in an executing program (PC, SP, Registers, Stack (local variables), Scheduling properties).
- **Shared State Among Threads (Process-Level):** The resources and memory shared by all threads residing in the same process (process identifiers (user ID, group ID, process ID), address space (program instructions, static data, heap and stack memory), and operating system resources (opened files, network sockets, and locks)).

C. Thread Operations

The program is split into multiple routines in order to execute in parallel. This execution results in either **true parallelism** (running simultaneously across multiple CPU cores) or **pseudo (interleaved) parallelism** (simulated on a single CPU core through rapid context-switching).

1) Thread Strategies:

- **Manager:** A single manager thread handles I/O, assigns work to worker threads.
- **Worker:** Worker threads can be created dynamically or allocated from a thread pool.
- **Pipeline:** Each thread handles a distinct stage of an assembly line, handing work to each other in a producer-consumer relationship

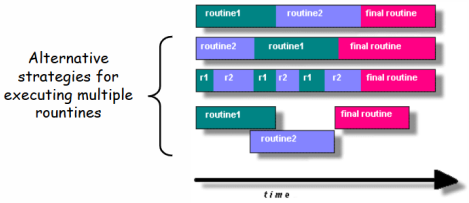


Fig. 16. Alternative Strategies for Executing Multiple Routines [13].

2) Pthreads API: `Pthreads` is a POSIX standard thread library. The first thread exists in `main()` and creates all subsequent threads.

Function	Description
`pthread_create()`	Creates a new thread executing the routine specified by `start_routine(arg)` and returns its thread ID in `thread`.
`pthread_exit()`	Terminates the calling thread and passes its return `status` to any joining thread.
`pthread_join()`	Blocks the calling thread until the thread specified by `threadid` terminates, enabling synchronization between threads.
`pthread_yield()`	Allows the calling thread to give up the CPU and re-enter the OS's run queue.

TABLE I
CORE PTHREADS MANAGEMENT APIS.

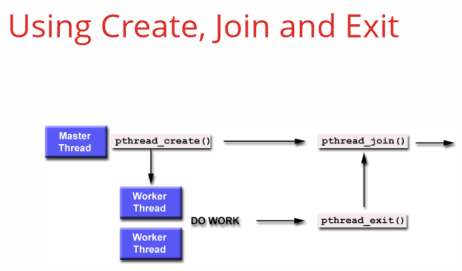


Fig. 17. Thread Lifecycle and Synchronization Workflow using `create()`, `join()`, and `exit()` [13].

D. Thread in Java

Java provides built-in multithreading support via the `Thread` class and the `Runnable` interface:

`Thread` class: The `Thread` class executes instructions from its `run()` method, which is implemented in a derived class. Calling `.start()` to get the thread ready to run, and the thread terminates when its `run()` method finishes.

```
class MyThread extends Thread {
    @Override
    public void run() {
        System.out.println("Thread " + Thread.currentThread
     ().getId() + " is running");
    }
}

public class Main {
    public static void main(String[] args) {
        Thread t = new MyThread();
        t.start(); // Call .start() to get the thread ready
      to run
    }
}
```

Listing 1. Creating a thread by extending the `Thread` class.

`Runnable` interface: Implementing the `Runnable` interface provides the `run()` method implementation, which is then then passed to a `Thread` object to execute.

```
class MyTask implements Runnable {
    @Override
    public void run() {
        System.out.println("Task running on Thread " +
     Thread.currentThread().getId());
    }
}

public class Main {
    public static void main(String[] args) {
        Thread t = new Thread(new MyTask());
        t.start();
    }
}
```

Listing 2. Creating a thread by implementing the `Runnable` interface.

These are some core Java `Thread` methods:

Method	Description
`.start()`	Begins a thread running by scheduling it to execute `run()` asynchronously.
`wait() / notify()`	Used for inter-thread synchronization and conditional coordination.
`.stop()`	Kills a specific thread (deprecated).
`.suspend() / .resume()`	Suspends and resumes thread execution (deprecated).
`.join()`	Waits for a specific thread to finish execution before continuing.
`.setPriority()`	Sets thread priority from 1 to 10 (`MIN_PRIORITY` to `MAX_PRIORITY`); default is 5 (`NORM_PRIORITY`).

TABLE II
CORE JAVA THREAD METHODS.

1) Thread Safety in Java Libraries: In the standard Java library, not all classes are thread-safe:

- **Thread-Safe:**
  - Most classes in `java.lang` (e.g., immutable types like `String`, `Integer`).
  - `java.util.Random` (designed for safe concurrent number generation).
  - `System.out` (safe concurrent output streams).
- **NOT Thread-Safe:**
  - Most standard collections in `java.util` (`ArrayList`, `HashMap`, `LinkedList`, `HashSet`). Multiple threads modifying an `ArrayList` simultaneously can corrupt its internal arrays.
  - Java GUI frameworks (`javax.swing`, `java.awt`). Modifying GUI components from background threads may lead to invalid visual state.

2) Atomicity: An operation is atomic if it executes as indivisible steps. In Java, integer increment (`count++`) is not atomic. It consists of three seperate operations:

1. Load data from variable `i`
2. Increment data by 1
3. Store data to variable `i`

3) Thread Pools: A program that creates a huge number of short-lived threads can be inefficient since threads are managed by the OS, which introduces significant memory overhead. The cost can be reduced by using **thread pool** which pre-allocate a fixed number of worker threads and keep them alive to execute tasks repeatedly.

A notable example of thread pool is in client-server architecture such as web servers, particularly where a high volume of short-lived client requests arrive concurrently. To avoid the overhead of creating a new thread for each connection, the server uses a fixed pool of worker threads to process incoming requests efficiently without exhausting system resources.

Thread pool works by maintaining a task queue with a fixed set of pre-allocated worker threads. When the queue is empty, worker threads sleep via `wait()`. When a new task arrives, an idle worker thread will be notified to dequeue and execute it. After that task is completed, the worker thread will return to the pool to execute subsequent tasks or wait for new tasks.

Thread Pools Java API: In Java, thread pools are provided by the `java.util.concurrent` package, with `ThreadPoolExecutor` serving as the implementation of the `ExecutorService` interface. Its core methods include:

- `execute()`: Creates a new thread inside the `ExecutorService` and returns immediately from each invocation.
- `shutdown()`: Initiates an orderly shutdown in which previously submitted tasks are executed, but no new tasks will be accepted.
- `awaitTermination()`: Waits for submitted tasks to finish. The program will not terminate until its last thread completes execution .

```java
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class ThreadPoolExample {
    public static void main(String[] args) throws
     InterruptedException {
        // Create a fixed thread pool with 3 worker threads
        ExecutorService executor = Executors.
     newFixedThreadPool(3);

        // Submit tasks to the executor
        executor.execute(new Task());
        executor.execute(new Task());
        executor.execute(new Task());

        // Orderly shutdown and wait for tasks to finish
        executor.shutdown();
        executor.awaitTermination(1, TimeUnit.MINUTES);
    }
}
```

Listing 3. Thread Pool implementation using `ExecutorService`.

Lab 4 - Part II

(1): Example 1

```
C:\Users\Admin\.jdks\openjdk
Main Thread Id: 3
Thread id: 27
Thread id: 28
Thread id: 29
Thread 3 prints 0
Thread 3 prints 1
Thread 3 prints 2
Thread 3 prints 3
Thread 29 prints 0
Thread 29 prints 1
Thread 27 prints 0
Thread 28 prints 0
Thread 3 prints 4
Thread 3 prints 5
Thread 29 prints 2
Thread 27 prints 1
Thread 28 prints 1
Thread 28 prints 2
```

Fig. 18. Console output from running `Thread1.java` (`example1`).

```java
Thread myThreads[] = new Thread[10];
for (int i=0; i<10; i++){
    myThreads[i] = new Thread1();
    myThreads[i].setName("T"+i);

    //set priority here
    myThreads[i].setPriority(i < 5 ? Thread.MIN_PRIORITY : Thread.MAX_PRIORITY);

    myThreads[i].start();
}
```

Fig. 19. Setting thread priorities in `Thread1.java` (`example1b`).

Thread priority is configured using `setPriority()` before invoking `start()`. It range from `Thread.MIN_PRIORITY` (which is 1, the lowest scheduling preference) to `Thread.MAX_PRIORITY` (which is 10, the highest scheduling preference) [14].

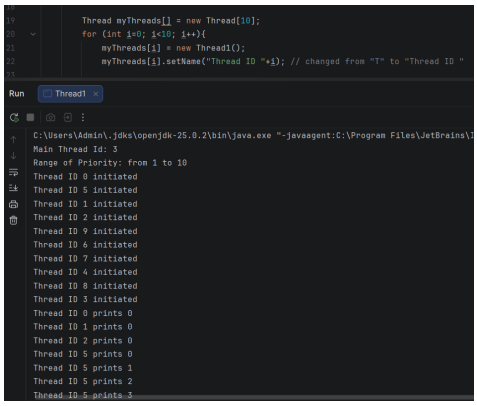


Fig. 20. Changed the thread name using `setName()` in `Thread1.java` (`example1b`).

A thread's name can be configured using the `setName()` method.

(2): Example 2

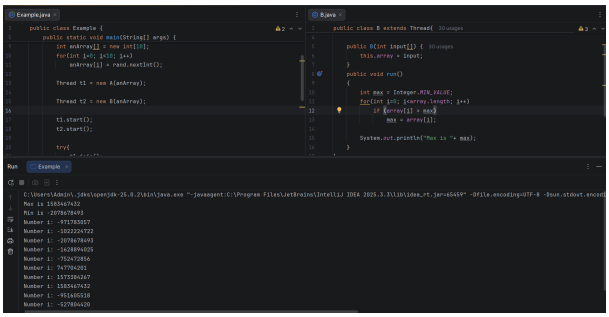


Fig. 21. Implementation and execution of `example2b`.

(3): Example 3

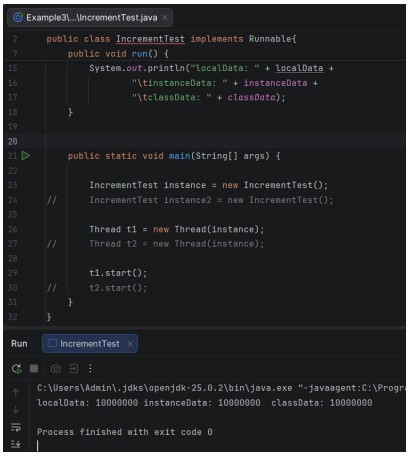


Fig. 22. Execution of a single thread `t1` (`example3`).

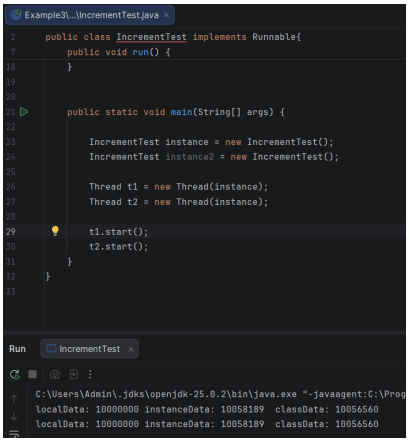


Fig. 23. Concurrent execution of two threads `t1` and `t2` (`example3`).

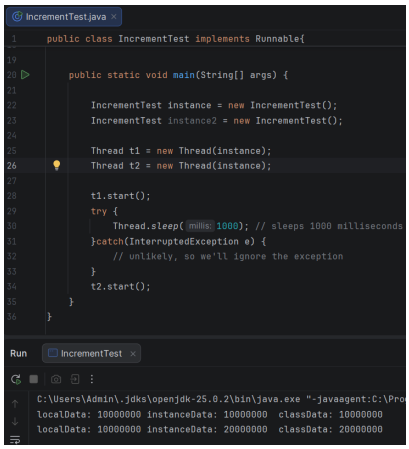


Fig. 24. Concurrent execution of two threads `t1` and `t2` with `Thread.sleep(1000)` (`example3b`).

As `Thread.sleep(1000)` pauses added to the main thread which allows `t1` to finish completely before `t2` starts. Two threads running sequentially will prevent race conditions, so both `instanceData` and `classData` correctly reach `20000000`.

References

[1] R. Pike, "Concurrency is not Parallelism," presented at *Heroku Waza*, San Francisco, CA, USA, 2012. [Online]. Available: https://go.dev/talks/2012/waza.slide#8.

[2] TechDifferences, "Difference Between Concurrency and Parallelism," *TechDifferences*, [Online]. Available: https://techdifferences.com/difference-between-concurrency-and-parallelism.html.

[3] D. Rákos, "SIMD in the GPU world," *RasterGrid Blog*, Feb. 2022. [Online]. Available: https://www.rastergrid.com/blog/gpu-tech/2022/02/simd-in-the-gpu-world/.

[4] GeeksforGeeks, "What is a Distributed System? Architecture and Components," *GeeksforGeeks*, [Online]. Available: https://www.geeksforgeeks.org/computer-networks/what-is-a-distributed-system/.

[5] CERN, "The Worldwide LHC Computing Grid (WLCG)," *CERN*, [Online]. Available: https://wlcg.web.cern.ch/.

[6] Wikipedia, "Cloud Computing," *Wikimedia Commons*, [Online]. Available: https://en.wikipedia.org/wiki/Cloud_computing#/media/File:Cloud_computing.svg. [Accessed: Sep. 27, 2026].

[7] Z. Ning, J. Huang, and X. Wang, "Vehicular Fog Computing: Enabling Real-Time Traffic Management for Smart Cities," *IEEE Wireless Communications*, vol. 26, no. 1, pp. 87–93, Feb. 2019, doi: https://doi.org/10.1109/MWC.2019.1700441.

[8] Mad Devs, "Everything You Need to Know About Edge Computing," *Mad Devs Blog*, [Online]. Available: https://maddevs.io/blog/everything-you-need-to-know-about-edge-computing/. [Accessed: Sep. 27, 2026].

[9] Swinburne University of Technology, "Lecture 2 - Process," *COS40003 Concurrent Programming*, 2026.

[10] E. W. Dijkstra, "The structure of the 'THE'-multiprogramming system," *Communications of the ACM*, vol. 11, no. 5, pp. 341–346, May 1968, doi: https://doi.org/10.1145/363095.363143.

[11] Swinburne University of Technology, "Lecture 3 (a) - Scheduling," *COS40003 Concurrent Programming*, 2026.

[12] B. Barney, "POSIX Threads Programming," *Lawrence Livermore National Laboratory*, [Online]. Available: https://hpc-tutorials.llnl.gov/posix/.

[13] Swinburne University of Technology, "Lecture 4 (a) - Thread," *COS40003 Concurrent Programming*, 2026.

[14] Oracle, "The Java Tutorials: Concurrency," [Online]. Available: https://docs.oracle.com/javase/tutorial/essential/concurrency/